ÓPTICA COMUNAL

Sistema de Gestión Integral

Documentación Técnica para Continuación de Desarrollo

Proyecto de Graduación — .NET 8 / ASP.NET Core MVC

Versión: 1.0 | Fecha: Marzo 2026

1. Contexto General del Proyecto

Este es el sistema de gestión integral para Óptica Comunal, una óptica real en operación. Es un proyecto de graduación en desarrollo activo. El sistema cubre gestión clínica, facturación, inventario, reportes y mesa de ayuda. Este documento está diseñado para ser usado como contexto por desarrolladores y por herramientas de IA asistentes.

1.1 Stack Técnico

Componente	Tecnología
Framework	.NET 8 — ASP.NET Core MVC
Base de datos	SQL Server con Entity Framework Core (Code-First)
Autenticación	Cookie Authentication + RBAC por Claims
Seguridad	BCrypt para contraseñas
Frontend	Bootstrap 5, jQuery Unobtrusive Validation, Chart.js, Select2
Exportación	ClosedXML para Excel (.xlsx)
IDE / ORM	Visual Studio 2022, EF Core Migrations

1.2 Arquitectura — Clean Architecture en 3 Capas

Proyecto	Responsabilidad	Contenido principal
OC.Core	Dominio puro, sin dependencias externas	Entidades, Enums, IGenericRepository
OC.Data	Persistencia	AppDbContext, Migraciones, GenericRepository, Seeding
OC.Web	Presentación y aplicación	Controllers, Views, ViewModels, Services, Program.cs

1.3 Regla de Oro: Code-First

NUNCA se ejecutan scripts SQL manuales para modificar el esquema de la base de datos. El flujo obligatorio es:

- 1. Modificar la entidad en OC.Core/Domain/Entities/
- 2. Generar migración: Add-Migration NombreDescriptivo -Project OC.Data -StartupProject OC.Web
- 3. Revisar el archivo generado en OC.Data/Migrations/ antes de aplicar
- 4. Aplicar: Update-Database

Los scripts SQL solo son válidos para seeding de datos (DbInitializer), nunca para cambios de esquema.

2. Estado Actual de la Base de Datos

El AppDbContext en OC.Data/Context/AppDbContext.cs contiene los siguientes DbSets:

2.1 Entidades Implementadas

DbSet	Módulo	Notas clave
Roles, Sucursales	RRHH/Config	Rol tiene Nombre como claim de autenticación
Usuarios	Seguridad	Tiene RolId y SucursalId. Claim 'UserId' en cookie
Empleados	RRHH	Ligado a Sucursal y Usuario
Pacientes	Clínico	Login propio con Claim 'PacienteId'. Cedula max 9 chars
SolicitudesCitas, Citas	Clínico	EstadoCita: Pendiente, Confirmada, Cancelada, Atendida
Expedientes, ValoresClinicos, DocumentosExpediente	Clínico	Expediente ligado a Cita (1-a-1). ValorClinico tiene graduación óptica OD/OI
Proveedores, Pedidos, DetallePedidos, Productos	Inventario/Compras	Pedido = compra a proveedor, NO venta. Producto tiene Stock
Ventas, DetalleVentas	Facturación	Venta tiene SucursalId, UsuarioId, PacienteId, MetodoPago enum
Equipos, Tickets, ComentarioTickets	Mesa de Ayuda	Ticket tiene NumeroSeguimiento max 20 chars
EnviosNotificacion	Notificaciones	Ligado a Cita para recordatorios automáticos

2.2 Entidades Nuevas — Módulo de Ventas

Agregadas en esta iteración. Incluir en contexto al continuar el módulo de facturación:

Venta.cs

```
public int Id { get; set; }
public string NumeroFactura { get; set; }   // FAC-{año}-{id:D6}
public int PacienteId { get; set; }          // FK → Paciente
public int UsuarioId { get; set; }            // FK → Usuario (recepcionista)
public int SucursalId { get; set; }           // FK → Sucursal (del Usuario)
public int? ValorClinicoId { get; set; }      // nullable — solo si incluye receta
public MetodoPago MetodoPago { get; set; }    // Enum
public decimal Total { get; set; }
public string? Notas { get; set; }
public DateTime FechaVenta { get; set; }
```

DetalleVenta.cs

```
public int Id { get; set; }
public int VentaId { get; set; }
public int? ProductoId { get; set; }          // nullable — lentes son precio libre
public string DescripcionSnapshot { get; set; } // foto del nombre al vender
public int Cantidad { get; set; }
public decimal PrecioUnitario { get; set; }   // precio libre, lo pone el
recepcionista
public decimal Subtotal { get; set; }
```

MetodoPago.cs (Enum)

```
// OC.Core/Domain/Enums/MetodoPago.cs
Efectivo = 1, SINPE = 2, Tarjeta = 3
```

3. Autenticación y Claims

El sistema tiene DOS tipos de usuario con flujos de login separados. Esto es crítico para entender el acceso en controllers.

3.1 Usuarios del Sistema (Admin, Recepcion, Optometrista, Tecnico)

Login en: /Account/Login → redirige a /Home

Claims disponibles después del login:

- ClaimTypes.Name → usuario.Nombre
- ClaimTypes.Email → usuario.Correo
- ClaimTypes.Role → usuario.Rol.Nombre ← usado en [Authorize(Roles = "X")]
- "UserId" → usuario.Id.ToString() ← usar para FK en entidades

Para obtener el ID del usuario logueado en un controller:

```
var usuarioIdStr = User.FindFirstValue("UserId");
int.TryParse(usuarioIdStr, out int usuarioId);
```

3.2 Pacientes

Login en: /PacienteAccount/Login → redirige a /PacienteDashboard

Claims disponibles:

- ClaimTypes.Role → "Paciente"
- "PacienteId" → paciente.Id.ToString()
- "Cedula" → paciente.Cedula

3.3 Roles Existentes

Rol	Acceso / Responsabilidad
Admin	Acceso total. Único rol que ve Reportes Gerenciales
Recepcion	Ventas, citas, pacientes, solicitudes
Optometrista	Expedientes, valores clínicos, historial clínico
Tecnico	Mesa de ayuda (tickets, equipos)
Paciente	Portal propio. No accede al sistema interno

4. Patrón de Repositorio

El sistema usa un único IGenericRepository<T> para todas las entidades. NO hay repositorios específicos por entidad salvo casos puntuales registrados en Program.cs.

4.1 Métodos Disponibles en IGenericRepository<T>

```
Task<PagedResult<T>> GetPagedAsync(int page, int size,
    Expression<Func<T, bool>>? filter = null,
    string? includeProperties = null);

Task<T?> GetByIdAsync(int id);
Task AddAsync(T entity);
Task UpdateAsync(T entity);
Task DeleteAsync(int id);
```

4.2 Uso del includeProperties

Para cargar navegaciones (FK) usar el parámetro includeProperties con nombres separados por coma. Para navegaciones anidadas usar punto:

```
// Una navegación
includeProperties: "Paciente"

// Varias navegaciones
includeProperties: "Paciente,Usuario,Sucursal"

// Navegación anidada
includeProperties: "Detalles.Producto"

// Compleja — módulo de ventas
includeProperties: "Paciente,Usuario,Sucursal,Detalles.Producto"
```

4.3 Registro en Program.cs

El registro genérico cubre todas las entidades automáticamente:

```
builder.Services.AddScoped(typeof(IGenericRepository<>),
typeof(GenericRepository<>));
```

No es necesario registrar IGenericRepository<Venta>, IGenericRepository<Cita>, etc. individualmente. Solo se registran explícitamente en casos especiales ya existentes (ValorClinico, DetallePedido, DocumentoExpediente).

6. Módulo de Facturación — Detalles de Implementación

6.1 Flujo de Negocio

- El recepcionista selecciona un paciente del dropdown (Select2, carga todo el inventario sin mínimo de caracteres)
- Si el paciente tiene ValorClinico, el sistema lo detecta via AJAX y ofrece incluir la receta óptica en la factura
- El recepcionista agrega productos del inventario (busca por nombre/SKU) o agrega lentes con precio libre
- Los lentes tienen campo de tecnología (Monofocal, Bifocal, Progresivo, Fotosensible, etc.) y precios separados para lente y aro
- El precio de TODOS los ítems lo define el recepcionista manualmente — no hay precio fijo en el sistema
- Al confirmar: se valida stock, se crea la Venta, se decrementa stock de productos con ProductoId, se redirige a la factura

6.2 Lógica de SucursalId en Venta

La sucursal de la venta se obtiene del Usuario logueado, NO de un selector en el formulario:

```
// En VentasController.cs — Create POST
var usuarioRes = await _usuarioRepo.GetPagedAsync(1, 1, u => u.Id == usuarioId);
var usuarioActual = usuarioRes.Items.FirstOrDefault();
venta.SucursalId = usuarioActual.SucursalId;
```

6.3 Lógica del Carrito

El carrito vive en JavaScript (array en memoria). Viaja al servidor serializado como JSON en un campo hidden:

```
// DetallesJson: campo hidden en el form
// DetalleVentaInputModel tiene ProductoId nullable
```

ProductoId es null para lentes (precio libre). El servidor distingue y solo decrementa stock cuando ProductoId tiene valor.

6.4 Factura

La vista Factura.cshtml usa Layout = null — no tiene sidebar ni navbar. Es un documento de 210mm. El botón de imprimir usa window.print() con @media print CSS. Cero dependencias NuGet adicionales.

6.5 Factura Electrónica Hacienda (CR) — Estado

NO implementada. Pendiente para versión posterior. El sistema genera comprobantes internos. La integración real requiere: XML firmado digitalmente, certificado del Ministerio de Hacienda, endpoints de producción/sandbox, clave numérica de 50 dígitos.

7. Módulo de Reportes Gerenciales

7.1 Acceso

Solo el rol Admin puede acceder. ReportesController tiene [Authorize(Roles = "Admin")]. El SidebarViewComponent solo muestra el ítem de reportes a ese rol.

7.2 Reportes Disponibles

Reporte	Acción	Contenido
Ventas	/Reportes/Ventas	KPIs, tendencia por día (línea), método de pago (doughnut), por sucursal (barras), top 10 productos (barras horizontales)
Fidelización	/Reportes/Fidelizacion	Clasificación Nuevo/Regular/Frecuente/Esporádico por período y sucursal. Pie chart + tabla filtrable
Demanda	/Reportes/Demanda	Citas agendadas vs atendidas vs canceladas. Barras apiladas por mes, tasas de atención/cancelación

7.3 Lógica de Fidelización

La clasificación se calcula en base al período seleccionado por el gerente:

- Nuevo: FechaRegistro del Paciente >= (fechaHasta - 1 mes)
- Frecuente: 4 o más citas Atendidas en el período
- Regular: 2 a 3 citas Atendidas en el período
- Esporádico: 1 cita Atendida, no es nuevo

Solo cuentan citas con Estado = EstadoCita.Atendida.

7.4 Exportación Excel (ClosedXML)

Cada reporte tiene su endpoint de exportación independiente que recibe los mismos parámetros de filtro que la vista:

```
GET /Reportes/ExportarVentasExcel?desde=&hasta=&sucursalId=
GET /Reportes/ExportarFidelizacionExcel?desde=&hasta=&sucursalId=
GET /Reportes/ExportarDemandaExcel?desde=&hasta=&sucursalId=
```

El NuGet requerido en OC.Web: ClosedXML (ya instalado).

7.5 Gráficas

Chart.js 4.4.1 cargado desde CDN en cada vista de reporte. Sin instalación local. Los datos viajan del servidor al JS como JSON serializado con System.Text.Json.JsonSerializer.Serialize().

8. Convenciones del Proyecto

8.1 Controllers

- Todos los controllers de sistema usan [Authorize] a nivel de clase
- Acciones públicas (registro de paciente, login) tienen [AllowAnonymous]
- Todos los POST tienen try/catch con TempData["Error"] y redirect limpio — nunca se muestra pantalla de excepción
- ViewBag se recarga en un método privado RecargarViewBag() para reutilizar en errores de POST

8.2 ViewModels

- Todos los formularios usan ViewModels, nunca entidades directamente en la vista
- Los ViewModels viven en OC.Web/ViewModels/
- Un archivo por ViewModel, o varios relacionados en un archivo (ej: ReportesViewModels.cs)

8.3 Vistas

- Razor (.cshtml). Comentarios con @* texto *@ — nunca <!-- --> para código Razor
- Bootstrap 5 para UI. La clase card-dashboard es el contenedor estándar de contenido
- Validaciones client-side con jQuery Unobtrusive Validation
- Para selects con búsqueda usar Select2 desde CDN (cdnjs.cloudflare.com)

8.4 Sidebar

El SidebarViewComponent en OC.Web/ViewComponents/Sidebar/ renderiza el menú según el rol. Para agregar un ítem:

```
menuItems.Add(new MenuItem {
    Title = "NombreModulo",
    Url   = "/NombreController",
    Icon  = "bi bi-nombre-icono"
});
```

Los iconos son de Bootstrap Icons (bi bi-*). Para agregar con restricción de rol:

```
if (rol == "Admin")
    menuItems.Add(new MenuItem { ... });
```

9. Historias de Usuario Pendientes — Guía para Continuar

9.1 OT-HU-021 a 023 — Órdenes de Trabajo

Contexto: gestión del proceso de fabricación/entrega de lentes. HU-023 requiere notificar al paciente cuando sus lentes están listos.

- Entidades nuevas sugeridas: OrdenTrabajo (Estado, PacienteId, CitaId?, FechaEstimada), DetalleOrden
- Estados sugeridos: Recibida, EnProceso, Lista, Entregada
- HU-023 puede reutilizar el sistema de notificaciones existente (EnvioNotificacion + INotificationService)
- El paciente puede ver el estado desde su dashboard (WEB-HU-032)

9.2 WEB-HU-028 a 032 — Portal del Paciente

El login de paciente ya está implementado en PacienteAccountController y PacienteDashboardController. Lo que falta:

- HU-028: Ya implementado (login seguro existe)
- HU-029: Ver citas (base existe en CitasPublicasController). Falta mostrar expedientes/tratamientos
- HU-030: Ver facturas propias — filtrar Ventas por PacienteId del claim
- HU-031: Agendar citas desde web — base existe en CitasPublicasController
- HU-032: Ver estado de lentes — depende de OT-HU-021 a 023

Para obtener PacienteId en el dashboard: User.FindFirstValue("PacienteId")

9.3 RH-HU-037 a 041 — Recursos Humanos

Módulo más complejo del backlog. Requiere nuevas entidades:

- HU-037: Datos laborales — extender Empleado o crear EntidadLaboral (salario, tipo contrato, fecha ingreso)
- HU-038: Horarios y asistencia — nuevas entidades Horario, RegistroAsistencia
- HU-039: Permisos/ausencias — entidad Ausencia (tipo, fecha, estado)
- HU-040: Cálculo de salarios — lógica de negocio en Service, considera CCSS, renta, otros (legislación CR)
- HU-041: Comprobante de pago — similar a Factura: vista sin layout, window.print()

Recomendación: abordar en orden 037 → 038 → 039 → 040 → 041. Cada HU depende de la anterior.

10. Errores Comunes y sus Soluciones

Error	Solución
FK constraint al migrar con datos existentes	Si son datos de prueba: DELETE FROM tabla. Si son reales: agregar SQL de UPDATE en la migración antes de crear el FK
JSON deserialize: cannot convert null to int	El campo en el InputModel debe ser int? (nullable), no int
Call is ambiguous between methods	Hay dos métodos con el mismo nombre en el mismo controller. Borrar el duplicado
Navegación null en la vista	Falta agregar la navegación al includeProperties del GetPagedAsync
AddScoped registrado dos veces	En Program.cs, el genérico ya cubre todo. Borrar el duplicado
Comentarios Razor con //	Usar @* comentario *@ en .cshtml, nunca // ni <!-- -->